SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This System Analysis Document (SAD) introduces PODPilot's strategic and technical architecture as a continuation of the Product Requirement Document (PRD). PODPilot is a GLM 5.1-powered multi-agent AI system designed to serve as an autonomous business co-pilot for Malaysian print-on-demand (POD) sellers on Shopee and Etsy.

1.1. Purpose:

This SAD highlights the technical scope and design decisions behind PODPilot's development. The key elements covered are:

- *Architecture: A cloud-hosted, service-oriented system with a Node.js backend, Next.js frontend, and five specialized AI agents coordinated by a GLM 5.1 Orchestrator, with Redis managing shared session state.*
- *Data Flows: How a seller's input (business idea, TikTok URL, complaint screenshot) flows from the frontend, through the Orchestrator, into the correct agent, and returns a structured output (listing, reply draft, P&L entry).*
- *Model Process: End-to-end workflows for each core feature — business onboarding, product design, listing launch, CS response, and financial tracking.*
- *Role of Reference: This document serves as the architectural reference for all developers, QA engineers, and evaluators to understand the system's design rationale and integration boundaries.*

1.2. Background:

© UMHackathon 2026

Malaysian POD sellers currently operate across disconnected tools — manually tracking TikTok trends, creating designs in Canva, copying product specs into Printify, responding to customer complaints one by one, and doing bookkeeping in spreadsheets. No single system coordinates these steps. PODPilot addresses this by unifying the full lifecycle under one stateful, AI-orchestrated session.

Previous Version / Changes in Major Architectural Components:

PODPilot is a net-new system (v1.0). There is no prior version. Key architectural innovations introduced in v1.0 include:

- *Stateful multi-agent session via Redis — agents share context without re-prompting the seller*
- *GLM 5.1 as a genuine decision-making layer (not just text generation) — used for pricing rationale, margin analysis, and severity classification*
- *Printify REST API integration with rate-limiting and queue management built in from day one*

1.3. Target Stakeholder

Stakeholder	*Role*	*Expectations*
POD Seller	*Primary user; submits prompts, approves designs, reviews CS replies*	*Minimal manual effort; fast listing creation; actionable financial insights*

© UMHackathon 2026

Printify	*Fulfilment partner; receives product specs and publishes to connected store*	*Correct API payloads; respect rate limits (200 req / 30 min)*
Etsy / Shopee	*Sales channels connected via OAuth*	*Well-formed listings; compliance with platform content policies*
Development Team	*Build and maintain PODPilot*	*Clear API contracts; modular agent codebase; Redis session schema documented*
QA Team	*Validate agent outputs and end-to-end flows*	*Defined input/output schemas per agent; edge case coverage for fallback states*

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web Application (Next.js frontend + Node.js backend), deployed on Vercel
Architecture	Service-oriented with dedicated AI agents, GLM 5.1 as a shared LLM service layer, Redis for distributed session state

PODPilot is structured as a client-server system with a clear separation of concerns. The Next.js (TypeScript) frontend renders the seller dashboard, product approval UI, CS panel, and finance view. It communicates exclusively with the Node.js backend via REST endpoints. The backend hosts the Orchestrator, which classifies every inbound request and dispatches it to one of five specialized agents. All agents share a common GLM 5.1 service interface via the Z.AI API, and their session state is persisted in Redis so that context carries seamlessly from the Onboarding Agent through to the Finance Tracker without requiring the seller to re-authenticate or re-enter data.

2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic "AI Module". The following Dependency diagram includes:

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.1.3 Dependency Diagram

a. *How the prompts are being constructed and sent to the GLM API*

b. *What goes into the context window?*

c. *How is your system receiving, parsing the responses and passing to the next system component?*

d. *Where the limitation of token are enforced or input are chunked before it actually reaches to GLm*

e. *Also, do outline all the major API calls between the system components [e.g. Customer Mobile App calls "Amazon API Gateway" for Order Service]*

f. *Do show how frontend, backend, services, database and external API interacts with each other [e.g. Order service communicates with Amazon RDS to ensure persistence of order data]*

g. *Must highlights the distributed components if the system is planning to be deployed across multiple services*

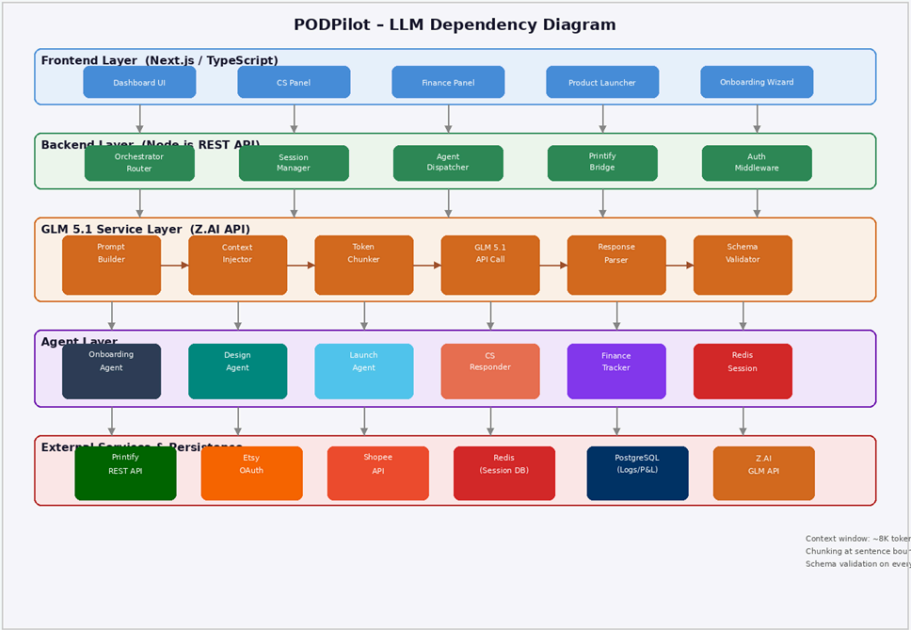

Figure 2.1: LLM Dependency Diagram - PODPilot System Layers

2.2.2. Sequence Diagram

a. *Here, do walk through one user interaction flow or customer journey with the stakeholders [e.g. Place Order Flow]*

b. *Show the interactions step by step.*

c. *Do demonstration of each key system features does operate from start to finish*

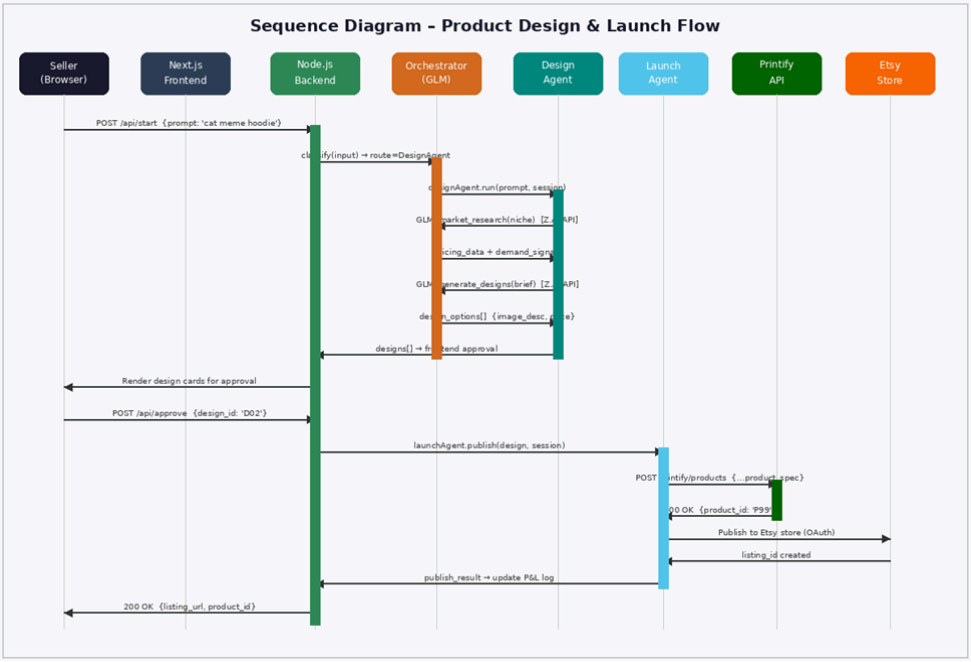

Figure 2.2: Sequence diagram - Product Design & Launch Flow

2.2. Technological Stack

Layer	Technology	Justification

Frontend	*Next.js 14 (TypeScript)*	*Server-side rendering for fast dashboard loads; TypeScript for type-safe agent payload contracts*
Backend	*Node.js (Express)*	*Non-blocking I/O suits high-concurrency agent dispatch; large npm ecosystem for Printify/Etsy SDK wrappers*
LLM	*GLM 5.1 via Z.AI API*	*Multimodal input support; strong JSON-mode for structured agent outputs; competitive cost per token*
Session State	*Redis*	*Sub-millisecond read/write for inter-agent state sharing; TTL-based session expiry; pub/sub for async agent signalling*
Database	*PostgreSQL*	*Relational model for P&L entries, agent logs, and seller records; 3NF design ensures data integrity*
Deployment	*Vercel*	*Zero-config Next.js deployment; serverless functions for backend; global CDN for frontend assets*
POD Fulfilment	*Printify REST API*	*Direct product creation and publishing; supports Etsy and Shopee as connected stores*
Sales Channels	*Etsy (primary), Shopee (secondary)*	*Largest POD-friendly marketplaces accessible to Malaysian sellers*

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.3. Key Data Flows

2.3.1. *Here, do show the how data are moving through the system and how its structure and stored*

2.3.1.1. Data Flow Diagram (DFD)

The DFD below illustrates data movement across the entire PODPilot system. External entities (Seller, Etsy/Shopee, Z.AI GLM API, Printify API) interact with seven core processes. The Redis Session Store and PostgreSQL database act as persistence layers. Data integrity rules: every process that writes to PostgreSQL must also update the Redis session to avoid Black Hole anomalies (data enters but never exits a process).

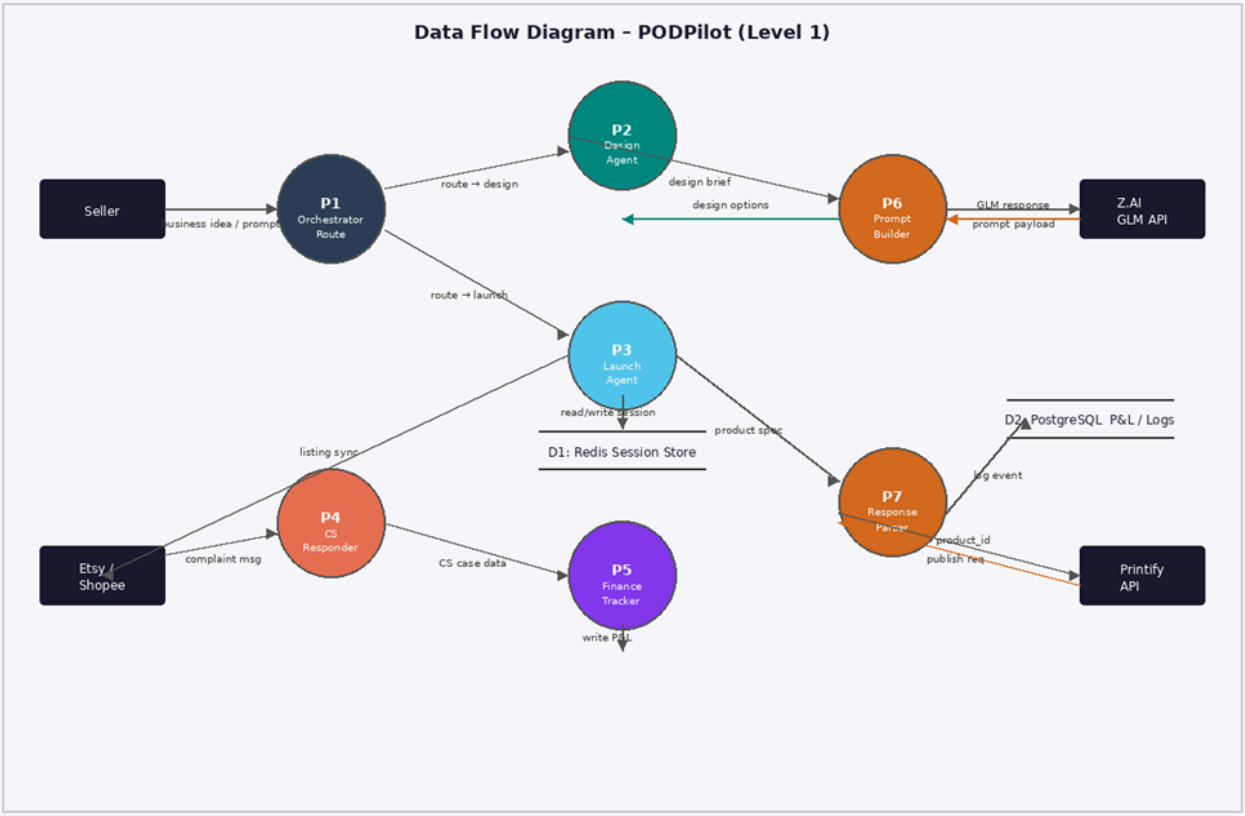

Figure 2.3: Data Flow Diagram (Level 1) - PODPilot

2.3.2. Normalized Database Schema

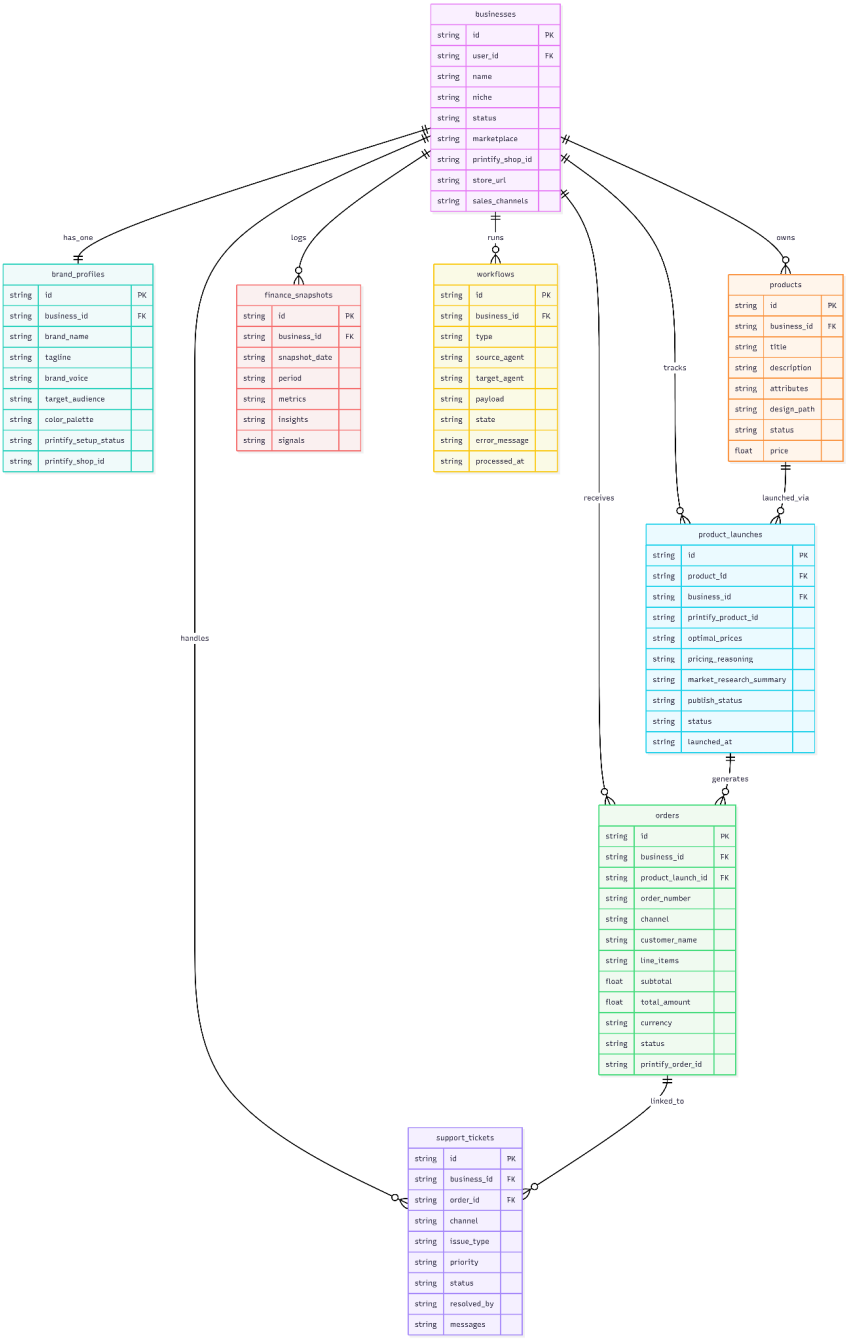

Figure 2.4: Normalized Database Schema (3NF ERD)

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Functional Requirements & Scope

This section defines the core boundaries of PODPilot's MVP, focusing on high-impact features that demonstrate technical feasibility within the UMHackathon 2026 timeline.

3.1. Minimum Viable Product:

#	Feature	Description
1	Stateful Onboarding	Seller enters a business idea; GLM generates brand identity and creates a Printify account via API. Session state persists to subsequent agents.
2	AI-Driven Product Design	Design Agent performs market research (pricing benchmarks, demand signals) and generates product design descriptions with GLM. Seller approves in the dashboard.
3	Automated Listing Launch	Launch Agent publishes approved products to Printify and syncs to Etsy store via OAuth. Handles rate limiting and queuing automatically.
4	CS Complaint Responder	Seller uploads a complaint screenshot or pastes text; GLM classifies severity, drafts a tone-appropriate reply, and flags refund/reprint actions.
5	Finance Tracker & AI Recommendations	P&L is auto-logged after each product publish and CS resolution. GLM analyses margins and fires actionable recommendations to the dashboard.

© UMHackathon 2026

Email: umhackathon@um.edu.my

3.2. Non-Functional Requirements (NFRs)

Quality	Requirement	Implementation
Scalability	System must handle concurrent agent sessions without session bleed	Redis namespace isolation per seller_id; Vercel serverless auto-scaling
Reliability	Product publish must succeed or be explicitly queued — no silent failures	Printify API calls wrapped in retry logic (3 attempts, exponential backoff); failed jobs logged to PostgreSQL
Maintainability	Each agent must be independently deployable and testable	Agents implemented as isolated Node.js modules with defined input/output schemas; Jest unit tests per agent
Token Latency	GLM API calls must return within 5s at p95 under normal load	Async API calls with 5s timeout; user-facing loading spinner while awaiting response; fallback error state on timeout
Cost Efficiency	Average token cost per session must not exceed RM 0.20	Fixed-length system prompts; conversation history summarised after 3 agent turns; repeated market research queries cached in Redis for 24 hours
Security	OAuth tokens for Etsy/Shopee must be stored encrypted at rest	Tokens encrypted with AES-256 before storage in PostgreSQL; never passed to GLM context

© UMHackathon 2026

3.3. Out of Scope / Future Enhancements

- *TikTok Shop × Printify direct integration (US-only; not available to Malaysian sellers)*
- *Native iOS / Android mobile application*
- *Real-time TikTok trend ingestion via TikTok API (requires developer approval)*
- *Multi-user team collaboration within a single PODPilot account*
- *Automated social media posting or ad campaign creation*
- *Custom fulfilment partner integration beyond Printify (e.g., Printful, Gelato)*
- *In-app live chat between seller and end customer*
- *Promotional voucher or discount code management*

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

New agent capabilities (e.g., an improved pricing model in the Design Agent) are released to 5% of active sellers first. System metrics (GLM latency, publish success rate, seller approval rate) are monitored for 24 hours. If no anomalies are detected, the rollout expands to 25%, 50%, and 100% in successive 12-hour windows. A/B testing is applied to GLM prompt variants — two prompt strategies for the CS Responder (e.g., formal vs. empathetic tone defaults) are served 50/50 and the variant with higher seller acceptance rate is promoted.

4.1.2. Emergency Rollback (ER) & Golden Release:

New agent capabilities (e.g., an improved pricing model in the Design Agent) are released to 5% of active sellers first. System metrics (GLM latency, publish success rate, seller approval rate) are monitored for 24 hours. If no anomalies are detected, the rollout expands to 25%, 50%, and 100% in successive 12-hour windows. A/B testing is applied to GLM prompt variants — two prompt strategies for the CS Responder (e.g., formal vs. empathetic tone defaults) are served 50/50 and the variant with higher seller acceptance rate is promoted.

4.1.3. Priority Matrix:

© UMHackathon 2026

Email: umhackathon@um.edu.my

Priority Matrix – PODPilot Incident Response

Priority	Trigger Condition	Threshold	Action	Owner
P1 Critical	GLM API error rate > 5% or Printify publish fails	3 consecutive failures	Emergency rollback to last stable Golden Release	Backend Lead
P2 High	Agent response latency > 5s at p95	p95 > 5000ms	Auto-scale Vercel functions; alert team	DevOps
P3 Medium	Redis session loss or cache miss > 30%	30% miss rate	Flush & rebuild cache; log for analysis	Backend Lead
P4 Low	Etsy listing sync failure	Single failure	Queue retry with exponential backoff	Backend Dev
P5 Info	Token cost per session exceeds RM 0.30	Single session	Log alert; review prompt compression	AI Engineer

Figure 4.1: Priority Matrix - PODPilot Incident Response

4.2. ***Monitoring***:

- *GLM API: response time (p50, p95, p99), error rate, token usage per session — monitored via custom middleware logging to PostgreSQL agent_logs table*
- *Printify API: publish success rate, rate-limit proximity (alert at 80% of 200 req/30 min quota)*
- *Redis: memory usage, session TTL expiry rate, cache hit/miss ratio — alert at >30% miss rate*
- *Vercel: function execution time, cold start frequency, error logs — monitored via Vercel Analytics dashboard*
- *Business metrics: average time from input to published listing; average seller approval rate for designs*

© UMHackathon 2026

4.3. *Assumptions:*

- *Sellers have a stable internet connection throughout their session*
- *Sellers have an active Etsy or Shopee seller account and complete the one-time OAuth connection during onboarding*
- *Printify account creation via API is available and not gated behind manual approval*
- *Z.AI GLM 5.1 API is available with sufficient rate limits for concurrent hackathon demo sessions*
- *All product designs are original and compliant with Etsy/Shopee content policies — the system flags but does not guarantee compliance*
- *Malaysian seller bank accounts and tax obligations are outside PODPilot's scope; the finance tracker reports revenue and cost only*

4.4. *External Dependencies:*

Tool / Service	*Purpose*	*Risk Level & Mitigation*
Z.AI GLM 5.1 API	***Core LLM engine — prompt inference, multi-step reasoning, structured output generation for all 5 agents***	***High — if GLM API is rate-limited or unavailable, core product fails. Mitigation: graceful degradation state shown to seller; retry with exponential backoff; cached responses used where available***
Printify REST API	***Product creation, variant management, and publishing to connected Etsy/Shopee store***	***High — publish failures block the Launch Agent. Mitigation: async queue with 3 retries; seller notified of delays via dashboard status***

© UMHackathon 2026

Etsy OAuth API	***Store connection and listing creation for the primary sales channel***	***Medium — OAuth token expiry breaks publishing. Mitigation: token refresh handled automatically; seller prompted to re-auth if refresh fails***
Shopee API	***Secondary sales channel listing sync***	***Medium — Shopee API changes may break integration. Mitigation: abstraction layer isolates Shopee logic; fallback to Etsy-only mode***
Redis Cloud	***Distributed session state and response caching across all agents***	***Medium — Redis downtime breaks cross-agent state. Mitigation: local in-memory fallback for single-session continuity; alert on connection failure***
Vercel	***Frontend and serverless backend deployment***	***Low — Vercel has 99.9% SLA. Mitigation: Golden Release rollback policy in place***

© UMHackathon 2026

5. *Project Management & Team Contributions*

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. *Project Timeline:*

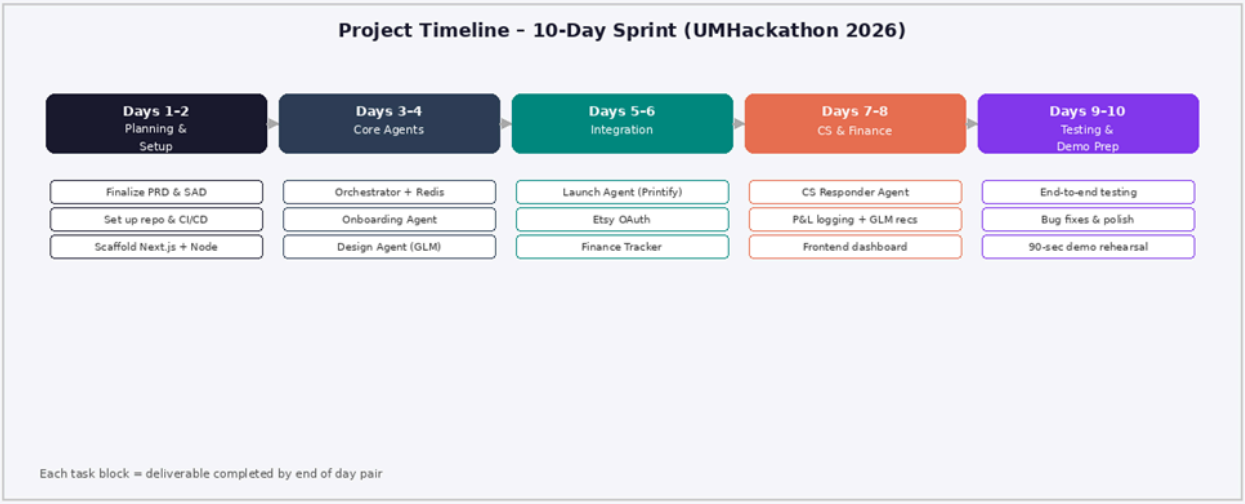

Figure 5.1: Project Timeline for UMHackathon 2026

© UMHackathon 2026

Email: umhackathon@um.edu.my

5.2. *Team Members Role:*

Member	*Role*	*Responsibilities*
Tan Jin Wei	*Full-Stack Lead*	*Next.js dashboard UI; REST API endpoints; Vercel deployment pipeline*
Harry Liow Siang Yi	*AI / Backend Engineer*	*GLM prompt engineering; Orchestrator routing logic; agent schema design; Redis session management*
Daniel Chong Chan Yip	*Integration Engineer*	*Printify REST API integration; Etsy OAuth flow; Launch Agent; rate-limit queue*
Ho Shuang Quan	*Data & Finance Engineer*	*PostgreSQL schema (3NF); Finance Tracker agent; P&L logging; GLM recommendation prompts*
Chew Xu Sheng	*QA & DevOps*	*End-to-end test cases; CI/CD pipeline; monitoring setup; demo rehearsal coordination*

5.3. *Recommendations:*

- *Redis Cache for market research: Cache GLM market research outputs per product niche for 24 hours to cut token costs by an estimated 40% for repeat queries in the same category*
- *Prompt compression after 3 agent turns: Summarise the conversation history with a single GLM call before passing to the next agent, keeping context window usage below 4K tokens for 95% of sessions*

© UMHackathon 2026

© UMHackathon 2026

Email: umhackathon@um.edu.my